LAPORAN

PENERAPAN TRANSAKSI DAN KONSISTENSI DATA PADA MYSQL

Disusun Untuk Memenuhi Tugas-3 P3

Mata Kuliah : Prak. Rekayasa & Arsitektur Basis Data

Dosen Pengampu:

Cecep Kurnia Sastradipraja, S.Kom., M.Kom.



Disusun Oleh:

Lisa Mutika Suganda 2507052

Kelas B

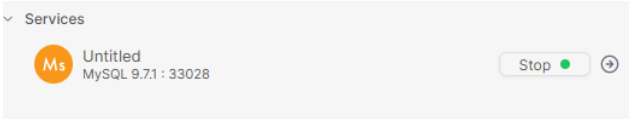
JURUSAN ILMU KOMPUTER

PROGRAM STUDI SISTEM INFORMASI

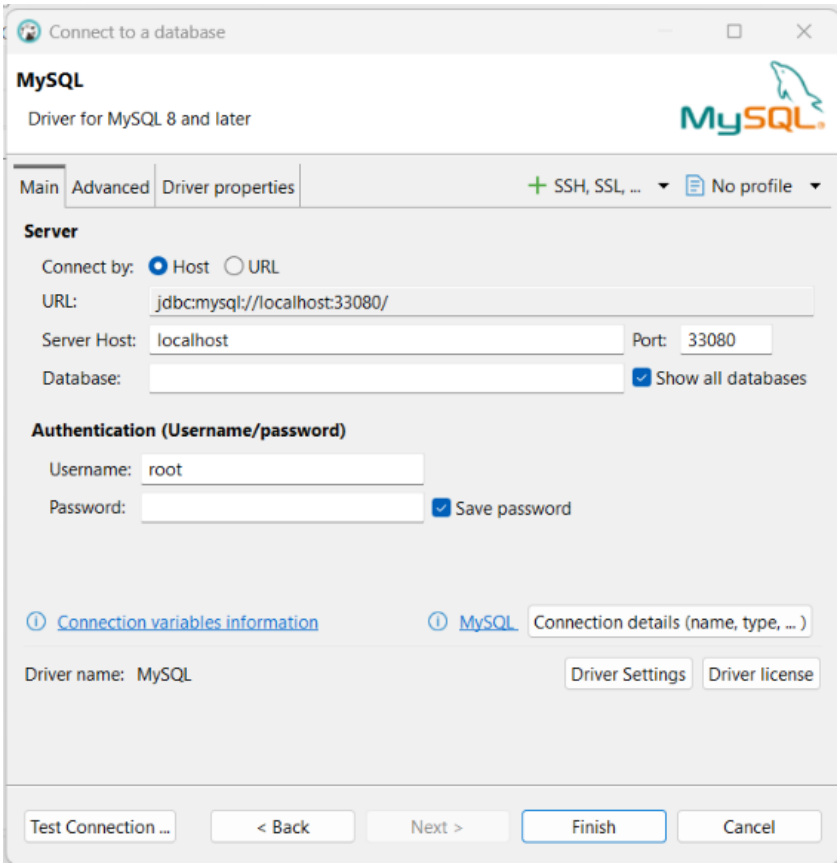
INSTITUT TEKNOLOGI GARUT

2026

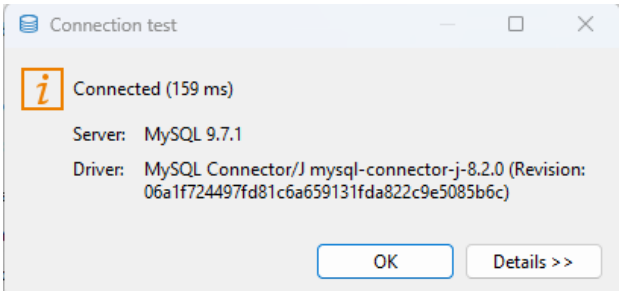
1. Mengaktifkan MySQL 9.7.1 di DBngin pada port 33080 hingga berstatus *Running*.



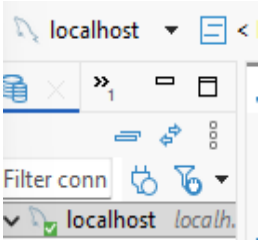
Membuat koneksi database MySQL baru di DBeaver dengan mengatur Server Host ke localhost dan Port 33080, selanjutnya menguji koneksi (*Test Connection*) hingga muncul status *Connected*.



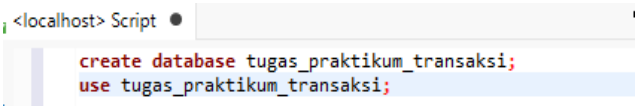
Jika sudah terkoneksi klik OK,



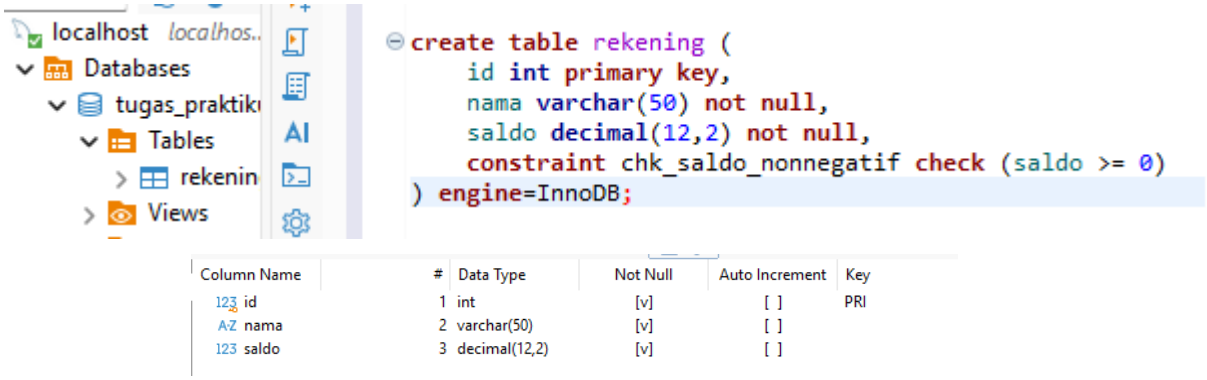
Klik kanan pada localhost, pilih SQL Editor, lalu klik New SQL Script untuk membuka editor QL



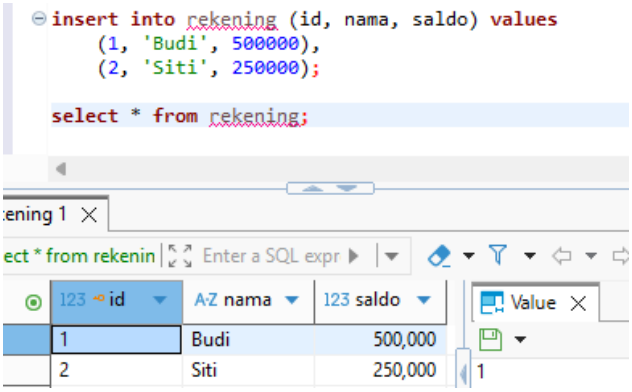
Buat database dengan menjalankan perintah create database tugas_praktikum_transaksi, kemudian aktifkan database menggunakan perintah use tugas_praktikum_transaksi



Membuat table rekening menggunakan engine innoDB dengan kolom id sebagai Primary key, nama, dan saldo, serta menambahkan Batasan CHECK agar saldo tidak negatif



Memasukkan data awal Budi dengan saldo Rp.500.000 dan Siti dengan saldo Rp.250.000 menggunakan perintah insert into, lalu memeriksa hasilnya dengan select * from rekening



2. Bagian I : Jawaban dari Latihan Mandiri

A. Transfer Berhasil

Memindahkan saldo Rp75.000 dari Budi ke Siti menggunakan **transaction.** Perubahan dissimpan secara permanen dengan perintah **commit**, saldo Budi menjadi Rp.425.000, Siti menjadi 325.000.

```sql
-- Latihan A: Transfer Berhasil (Rp75.000)
select * from rekening;

start transaction;

update rekening
set saldo = saldo - 75000.00
where ID = 1;

update rekening
set saldo = saldo + 75000.00
where ID = 2;

select * from rekening;

commit;

select * from rekening;
```

select * from rekening | Enter a SQL expression to filter resul

	id	nama	saldo
1	1	Budi	425,000
2	2	Siti	325,000

B. Batalkan Perubahan

Memulai transaksi baru dan mengurangi saldo Budi sebesar Rp20.000, saldo sementara Budi menjadi Rp.405.000 sebelum perubahan dibatalkan menggunakan peritah ROLLBACK.

```sql
-- Latihan B: Membatalkan Transaksi dengan ROLLBACK (Rp20.000)
select * from rekening;

start transaction;

update rekening
set saldo = saldo - 20000.00
where ID = 1;

select * from rekening;
```

select * from rekening | Enter a SQL exp

	id	nama	saldo
1	1	Budi	405,000
2	2	Siti	325,000

Menjalankan perintah **rollback** untuk membatalkan transaksi, sehingga saldo Budi menjadi Rp.425.000 dan saldo Siti tetap Rp.325.000.

```sql
rollback;

select * from rekening;
```

```
select * from rekening
```

	id	nama	saldo
1	1	Budi	425,000
2	2	Siti	325,000

C. SAVEPOINT

Memindahkan saldo Rp30.000 dari Budi ke Siti, lalu membuat savepoint bernama **setelah_transfer** sebagai penanda kondisi setelah transfer. Saldo budi menjadi Rp.395.000 saldo Siti Rp.335.000.

```
-- Latihan C: SAVEPOINT untuk pembatalan sebagian (Transfer Rp30.000, Tambahan Rp5.000)
start transaction;

update rekening
set saldo = saldo - 30000.00
where ID = 1;

update rekening
set saldo = saldo + 30000.00
where ID = 2;

SAVEPOINT setelah_transfer;

select * from rekening;
```

	id	nama	saldo
1	1	Budi	395,000
2	2	Siti	355,000

Selanjutnya, mengurangi kembali saldo Budi sebesar Rp5.000 untuk menguji pembatalan sebagian transaksi.

```
update rekening
set saldo = saldo - 5000.00
where ID = 1;
```

```
select * from rekening
```

	id	nama	saldo
1	1	Budi	390,000
2	2	Siti	355,000

Menjalankan perintah rollback to savepoint setelah_transfer untuk membatalkan pengurangan Rp.5.000, lalu menggunakan commit untuk menyimpan transfer Rp.30.000 secara permanen. Hasil akhirnya, saldo Budi Rp395.000 dan Siti Rp.355.000.

```
rollback to savepoint setelah_transfer;

commit;

select * from rekening;
```

id	nama	saldo
1	Budi	395,000
2	Siti	355,000

D. RELASE SAVEPOINT

Mengurangi saldo Budi sebesar Rp10.000 dalam transaksi, lalu membuat savepoint bernama sp_pemeriksaan.

```
-- Latihan D: Lepaskan savepoint dengan RELEASE SAVEPOINT
start transaction;

update rekening
set saldo = saldo - 10000.00
where ID = 1;

savepoint sp_pemeriksaan;

select * from rekening;
```

	id	nama	saldo
1	1	Budi	385,000
2	2	Siti	355,000

Selanjutnya, menambah saldo Siti sebesar Rp10.000, menghapus savepoint menggunakan perintah release savepoint sp_pemeriksaan, lalu menyimpan seluruh perubahan dengan commit.

```
update rekening
set saldo = saldo + 10000.00
where ID = 2;

release savepoint sp_pemeriksaan;

commit;

select * from rekening;
```

ning 1

:t * from rekening | Enter a SQL expression to filter results

123 id	A-Z nama	123 saldo
1	Budi	385,000
2	Siti	365,000

E. Analisis Risiko

Menurut saya, Tanpa transaksi eksplisit, jika sistem terhenti setelah saldo pengirim berkurang, saldo penerima belum tentu bertambah. Hal ini dapat menyebabkan data tidak seimbang dan transaksi tidak tercatat dengan benar.

3. Bagian II : Jawaban Evaluasi Pemahaman

1. **Apa yang dimaksud dengan transaksi pada basis data?**
2. **Apa perbedaan COMMIT dan ROLLBACK?**
3. **Mengapa transfer saldo sebaiknya menggunakan transaksi?**
4. **Apa fungsi SAVEPOINT?**
5. **Apa yang dilakukan ROLLBACK TO SAVEPOINT?**
6. **Apakah RELEASE SAVEPOINT sama dengan COMMIT? Jelaskan.**
7. **Mengapa tabel InnoDB digunakan dalam praktikum ini?**
8. **Apa yang perlu dilakukan jika hasil saldo tidak sesuai dengan hasil yang diharapkan?**

Jawab :

1. Transaksi adalah serangkaian perintah pada basis data yang dijalankan sebagai satu kesatuan untuk menjaga konsistensi data.
2. COMMIT digunakan untuk menyimpan perubahan secara permanen, sedangkan ROLLBACK digunakan untuk membatalkan perubahan yang belum di-commit.
3. Agar pengurangan saldo pengirim dan penambahan saldo penerima berjalan sebagai satu kesatuan, sehingga data tetap konsisten jika terjadi kesalahan.
4. SAVEPOINT digunakan untuk membuat titik simpan dalam transaksi agar perubahan dapat dibatalkan sebagian tanpa membatalkan seluruh transaksi.
5. Perintah ini digunakan untuk membatalkan perubahan setelah titik savepoint tertentu tanpa membatalkan perubahan sebelumnya.
6. Tidak. RELEASE SAVEPOINT menghapus titik simpan, sedangkan COMMIT menyimpan seluruh perubahan transaksi secara permanen.
7. Karena InnoDB mendukung transaksi, COMMIT, ROLLBACK, dan SAVEPOINT, serta membantu menjaga konsistensi data.
8. Memeriksa kembali perintah SQL dan hasil saldo. Jika transaksi belum di-commit, gunakan ROLLBACK untuk membatalkan perubahan yang salah, lalu ulangi transaksi dengan benar.